Implementasi *Face Detection* Menggunakan Algoritma HOG dan *Haarcascade*

Disusun dalam Rangka Memenuhi Tugas Ujian Tengah Semester Mata Kuliah Analisis Data Tidak Tersrtuktur

Semester Genap Tahun Akademik 2025/2026

Dosen Pengampu:

Abdul Kudus, S.SI., M.SI., PH.D.

Disusun Oleh:

Keysha Jauzi Althaff 10060122019

PROGRAM STUDI STATISTIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM (MIPA)

UNIVERSITAS ISLAM BANDUNG

2025

Lakukan deteksi wajah (*face detection*) terhadap gambar



yang disimpan di link https://bit.ly/dosenSTKUnisba dengan tahapan:

1. Baca file gambar **(CPMK01, SKOR 10)**
2. Lakukan preprocessing: **(CPMK03, SKOR 30)**
    a. Atur ulang ukuran resolusi gambar menjadi 800×600 piksel
    b. Konversi file gambar menjadi abu-abu (*grayscale*)
3. Lakukan deteksi wajah dengan: **(CPMK02, SKOR 40)**
    a. Algoritme Haarcascade. Berapa wajah yang terdeteksi?
    b. Algoritme *Histogram of Oriented Gradients* (HOG). Berapa wajah yang terdeteksi?
4. Dari hasil nomor 3, algoritme apa yang lebih akurat? Apakah Haarcascade atau HOG? **(CPMK04, SKOR 20)**

JAWAB

1. Membaca file gambar

```
[3] import cv2

[4] from google.colab import drive
    drive.mount('/content/drive')

    Mounted at /content/drive

[6] image = cv2.imread('/content/drive/MyDrive/ADTT/dosenSTK.jpg')

[7] image.shape

    (1200, 1800, 3)

[8] from google.colab.patches import cv2_imshow
    cv2_imshow(image)
```



0 d selesai pada 22.15

Interpretasi syntax per baris:

- Pada baris pertama kita mengimpor *library* OpenCV.
- Lalu, kita membuat google collab *connect* ke google drive.
- Kita membaca file gambar yang sebelumnya telah di upload di google drive, dengan menggunakan *function* cv2.imread() dan *copy path* gambar di google drive.
- image.shape digunakan untuk mengetahui resolusi gambar.
- Setelah itu kita tampilkan gambar menggunakan *function* cv2_imshow().

2. Melakukan preprocessing

a. Atur ulang ukuran resolusi gambar menjadi 800x600 piksel

```
[9] #a. Atur ulang ukuran resolusi gambar menjadi 800×600 piksel
    image = cv2.resize(image, (800, 600))
    image.shape

    (600, 800, 3)

    cv2_imshow(image)
```



1 m 53 d selesai pada 20.48

Interpretasi syntax per baris:

- Mengatur ulang resolusi gambar dengan menggunakan *function* cv2.resize dan mengecek resolusi lagi menggunakan *function* image.shape
- Setelah itu tampilkan gambar menggunakan *function* cv2_imshow().

b. Konversi file gambar menjadi abu-abu (grayscale)

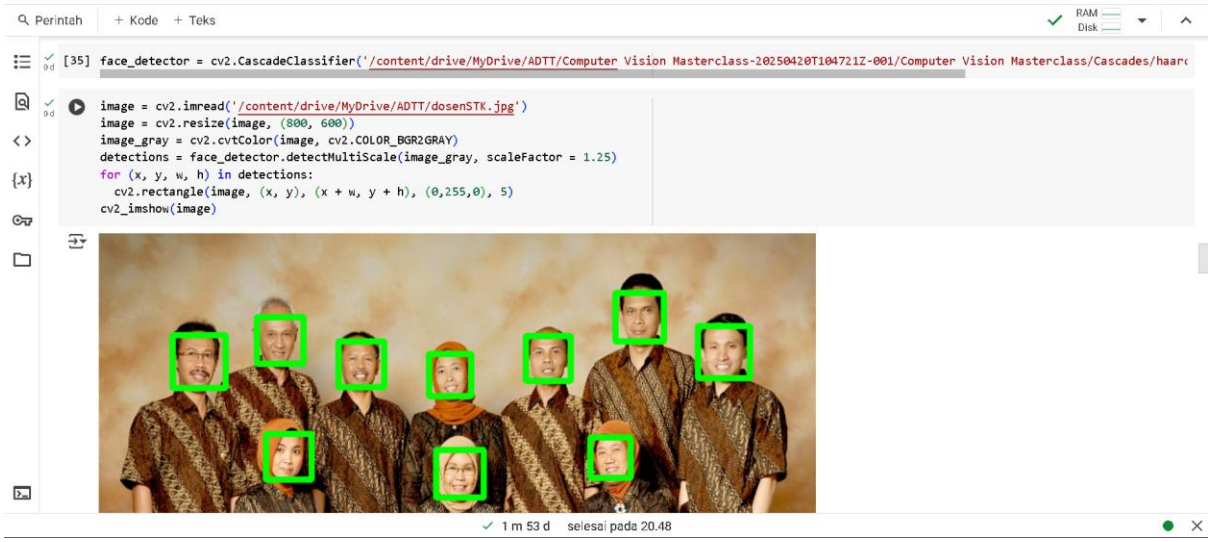


Interpretasi syntax per baris:

- Mengubah gambar menjadi abu-abu atau *grayscale* menggunakan *function* cv2.cvtColor() dengan cv2.COLOR_BGR2GRAY. Setelah itu tampilkan gambar menggunakan *function* cv2_imshow().

3. Deteksi Wajah

a. Algoritme Haarcascade. Berapa wajah yang terdeteksi?



Interpretasi:

Menggunakan *scale factor* sebanyak 1.25, terdeteksi ada 10 wajah pada gambar

Atau bisa lebih akurat lagi menggunakan *syntax* dibawah ini:

```
[35] face_detector = cv2.CascadeClassifier('/content/drive/MyDrive/ADTT/Computer Vision Masterclass-20250420T104721Z-001/Computer Vision Masterclass/Cascades/haarc

image = cv2.imread('/content/drive/MyDrive/ADTT/dosenSTK.jpg')
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
image = cv2.resize(image, (800, 600))
image_gray = cv2.resize(image, (800, 600))
detections = face_detector.detectMultiScale(image_gray, scaleFactor=1.25, minNeighbors=7,
                                            minSize=(20,20), maxSize=(100,100))
for (x, y, w, h) in detections:
  print(w, h)
  cv2.rectangle(image, (x, y), (x + w, y + h), (0,255,0), 2)
cv2_imshow(image)

54 54
50 50
52 52
58 58
59 59
51 51
47 47
45 45
53 53
51 51
```



```
0 d  selesai pada 21.53

[40] len(detections)

10
```

Interpretasi:

Syntax detections lebih akurat dengan menyantumkan ukuran yang tetap, sehingga terdekteksi terdapat 10 wajah pada gambar tersebut.

b. Algoritme Histogram of Oriented Gradients (HOG). Berapa wajah yang terdeteksi?

```
[28] import dlib

image = cv2.imread('/content/drive/MyDrive/ADTT/dosenSTK.jpg')
cv2_imshow(image)
```



0 d selesai pada 21.53

```
[30] face_detector_hog = dlib.get_frontal_face_detector()

[38] detections = face_detector_hog(image, 4)

[32] detections, len(detections)

(rectangles[[(199, 235) (306, 343)], [(1081, 245) (1171, 335)], [(613, 255) (703, 345)], [(842, 287) (917, 362)], [(862, 504) (952, 594)], [(1519, 226) (1609, 315)], [(404, 206) (494, 295)], [(1310, 146) (1400, 236)], [(1240, 464) (1330, 554)], [(434, 454) (524, 544)]],
 10)

for face in detections:
  #print(face)
  #print(face.left())
  #print(face.top())
  #print(face.right())
  #print(face.bottom())
  l, t, r, b = face.left(), face.top(), face.right(), face.bottom()
  cv2.rectangle(image, (l, t), (r, b), (0, 255, 255), 2)
cv2_imshow(image)
```



0 d selesai pada 21.53

```
[60] len(detections)

10
```

Interpretasi:

Menggunakan algoritma HOG terdeteksi 10 wajah pada gambar tersebut.

4. **Algoritma apa yang lebih akurat?**

   HOG lebih akurat dibandingkan Haarcascade karena tidak memerlukan pengaturan skala deteksi (scale rate) secara eksplisit. Hal ini menjadikan HOG lebih stabil dan konsisten dalam mengenali objek pada berbagai ukuran dan kondisi.

Lampiran Syntax Google Collab:

UTS ANALISIS DATA TIDAK TERSTRUKTUR

Nama: Keysha Jauzi Althaff

NPM: 10060122019

Kelas: A

1. Baca File Gambar

```
import cv2
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
image = cv2.imread('/content/drive/MyDrive/ADTT/dosenSTK.jpg')
```

```
image.shape
```

```
from google.colab.patches import cv2_imshow
cv2_imshow(image)
```



2. Lakukan Preprocessing:

- a. Atur ulang ukuran resolusi gambar menjadi 800×600 piksel
- b. Konversi file gambar menjadi abu-abu (grayscale)

```
#a. Atur ulang ukuran resolusi gambar menjadi 800×600 piksel
image = cv2.resize(image, (800, 600))
image.shape
```

```
(600, 800, 3)
```

```
cv2_imshow(image)
```



```
600 * 800 * 3, 600 * 800, 1440000 - 480000
```

```
(1440000, 480000, 960000)
```

```
#b. Konversi file gambar menjadi abu-abu (grayscale)
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2_imshow(image_gray)
```



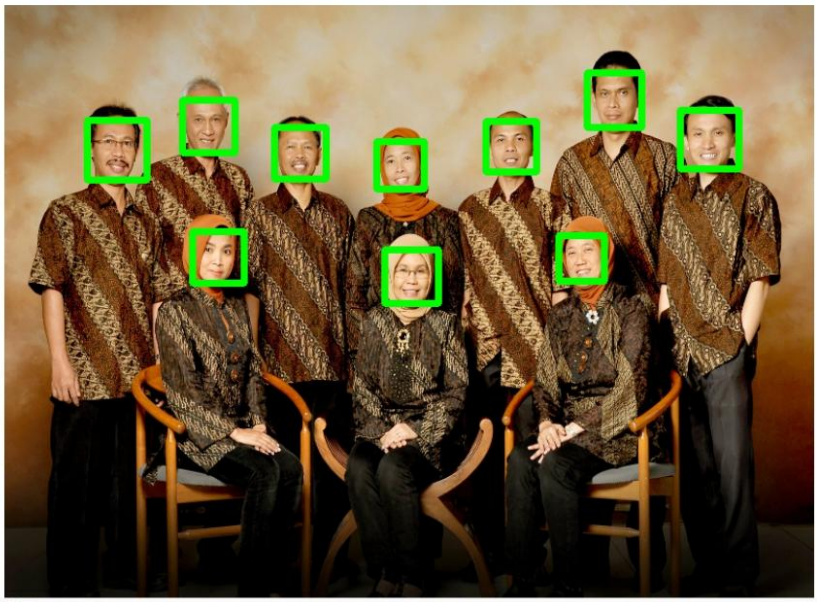
3. Lakukan Deteksi Wajah dengan:

- a. Algoritme Haarcascade. Berapa wajah yang terdeteksi?
- b. Algoritme Histogram of Oriented Gradients (HOG). Berapa wajah yang terdeteksi?

a.Algoritme Haarcascade. Berapa wajah yang terdeteksi?

```
face_detector = cv2.CascadeClassifier('/content/drive/MyDrive/ADTT/Computer Vision Masterclass-20250420T104721Z-001/Computer Visio
```

```
image = cv2.imread('/content/drive/MyDrive/ADTT/dosenSTK.jpg')
image = cv2.resize(image, (800, 600))
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
detections = face_detector.detectMultiScale(image_gray, scaleFactor = 1.25)
for (x, y, w, h) in detections:
  cv2.rectangle(image, (x, y), (x + w, y + h), (0,255,0), 5)
cv2_imshow(image)
```



```
len(detections)
```

```
10
```

b. Algoritme Histogram of Oriented Gradients (HOG). Berapa wajah yang terdeteksi?

```
import dlib
```

```
image = cv2.imread('/content/drive/MyDrive/ADTT/dosenSTK.jpg')
image = cv2.resize(image, (800, 600))
cv2_imshow(image)
```



```
face_detector_hog = dlib.get_frontal_face_detector()
```

```
detections = face_detector_hog(image, 1)
```

```
detections
```

```
for face in detections:
  #print(face)
  #print(face.left())
  #print(face.top())
  #print(face.right())
  #print(face.bottom())
  l, t, r, b = face.left(), face.top(), face.right(), face.bottom()
  cv2.rectangle(image, (l, t), (r, b), (0, 255, 255), 2)
cv2_imshow(image)
```

